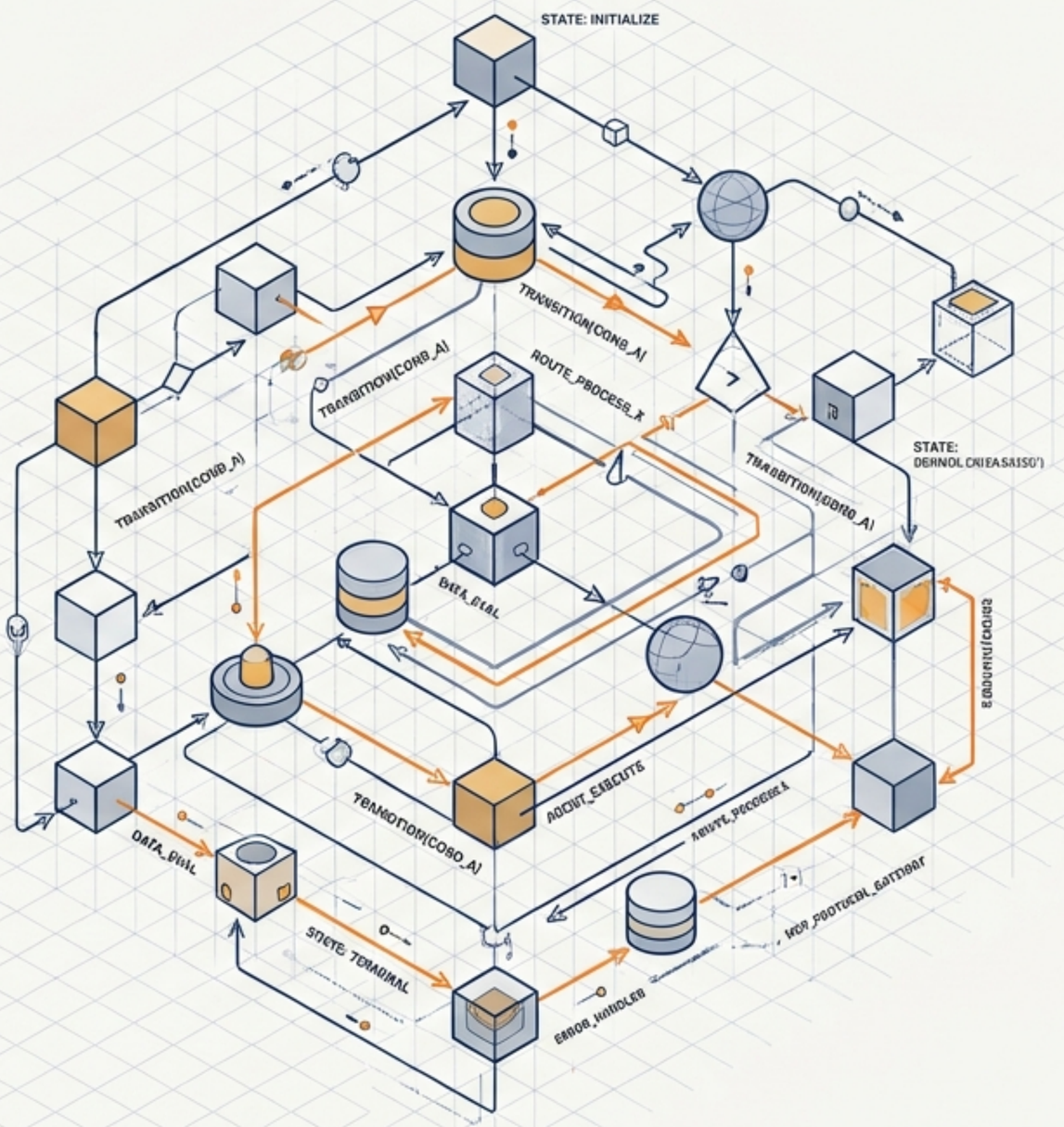
Engenharia de Agentes de IA

Arquitetura, Execução Prática e Avaliação de Sistemas Autônomos

- **Público:** Arquitetos, Engenheiros de Software e Dados.
- **Escopo:** Do design conceitual à execução segura em produção.
- **Foco:** Modelos mentais, implementação Python (LangGraph) e Protocolos (MCP).



Gemini Notebook

A Mudança de Paradigma: Além do Turno Único

Chatbots (Passado)



Operação

Reativos (pergunta-resposta).



Estado

Stateless (limitados à janela de contexto atual).

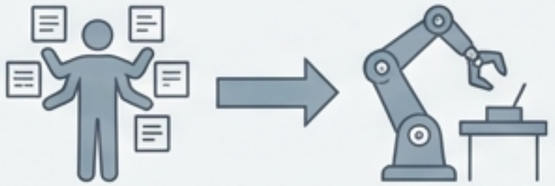


Saída (Output)

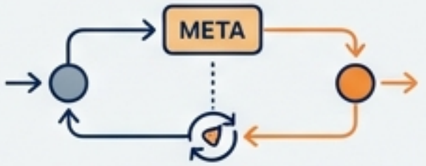
Geram apenas texto ou código.

Delegação

Você terceiriza a tarefa (micro-gerenciamento).

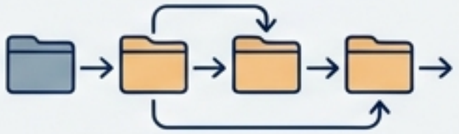


Agentes (Presente/Futuro)



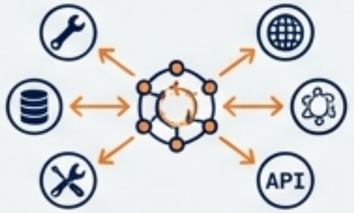
Operação

Proativos (orientados a metas em malha fechada).



Estado

Stateful (mantêm memória persistente e contínua).

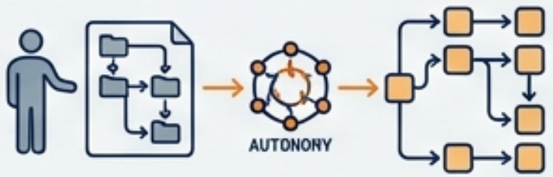


Saída (Output)

Executam ações no mundo real via invocação de ferramentas.

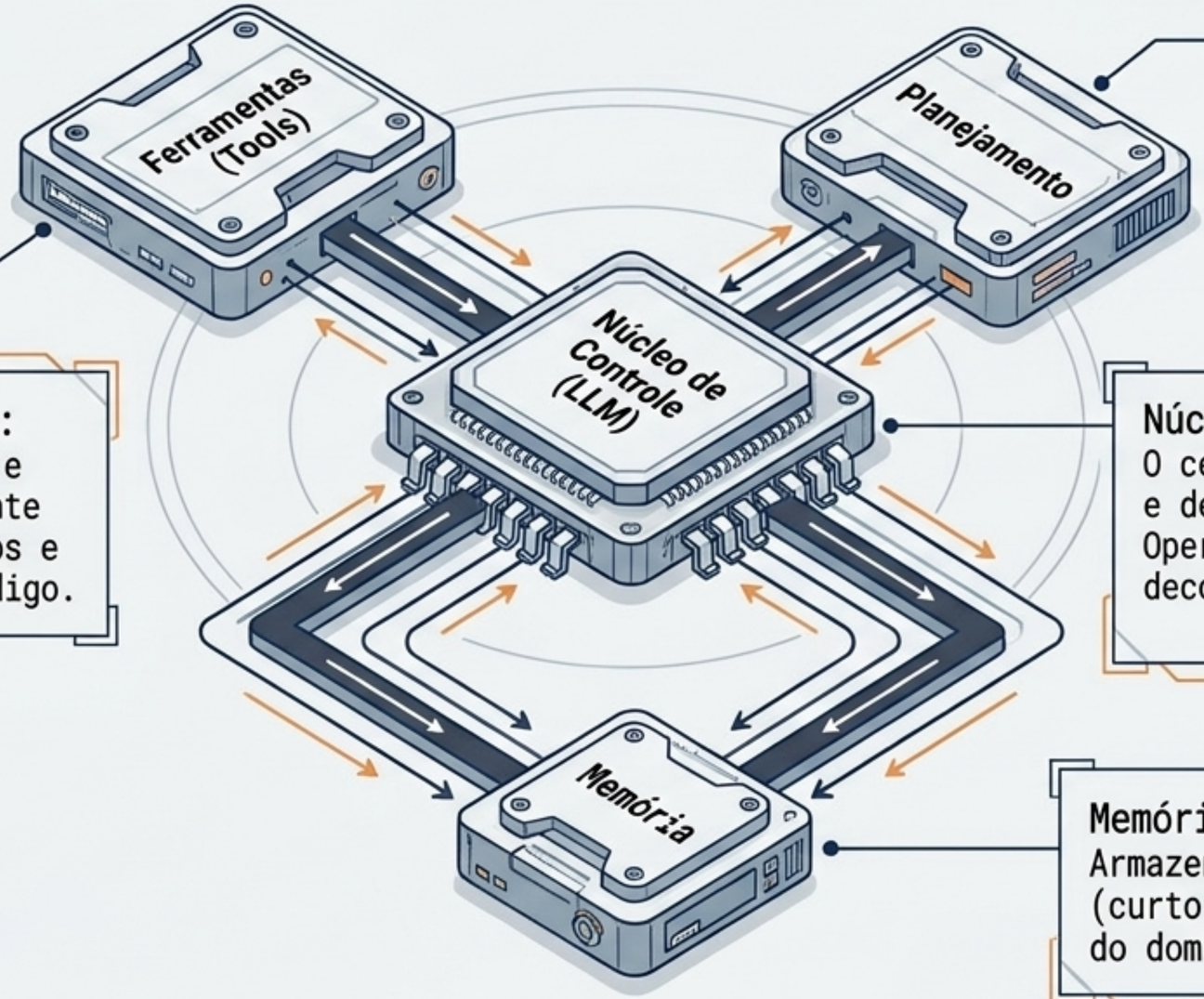
Delegação

Você terceiriza o processo inteiro (autonomia).



Gemini Notebook

A Anatomia de um Agente Autônomo



Ferramentas (Tools):
A interface sensorial e motora. Conecta o agente a APIs, Bancos de Dados e interpretadores de código.

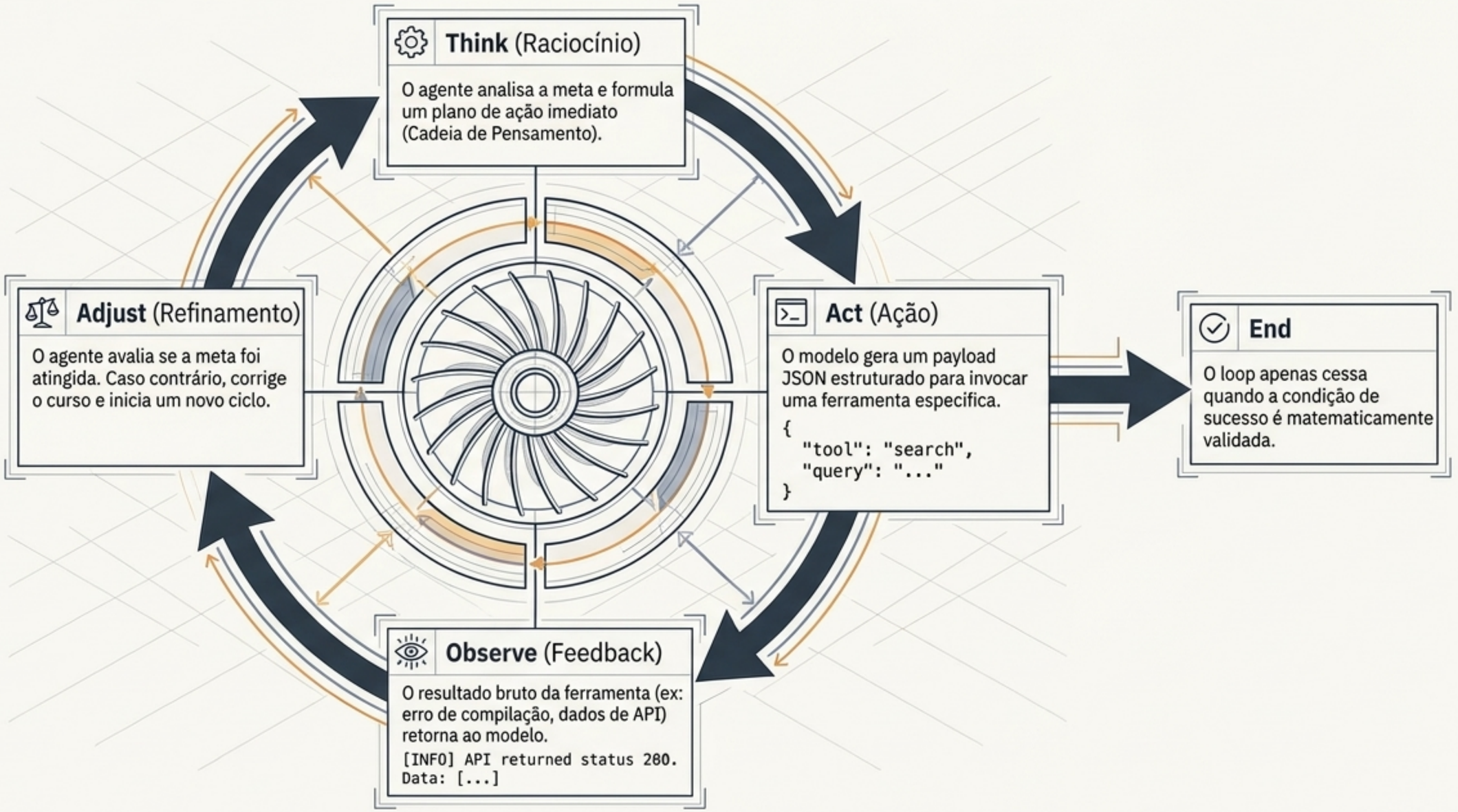
Planejamento:
Decompõe metas complexas em passos acionáveis, gerenciando o raciocínio lógico.

Núcleo de Controle (LLM):
O cérebro. Processa intenções e decide a próxima ação. Opera com temperatura=0 para decodificação determinística.

Memória:
Armazena o histórico do estado (curto prazo) e recupera fatos do domínio (longo prazo).

Gemini Notebook

O Motor de Execução: O Padrão ReAct

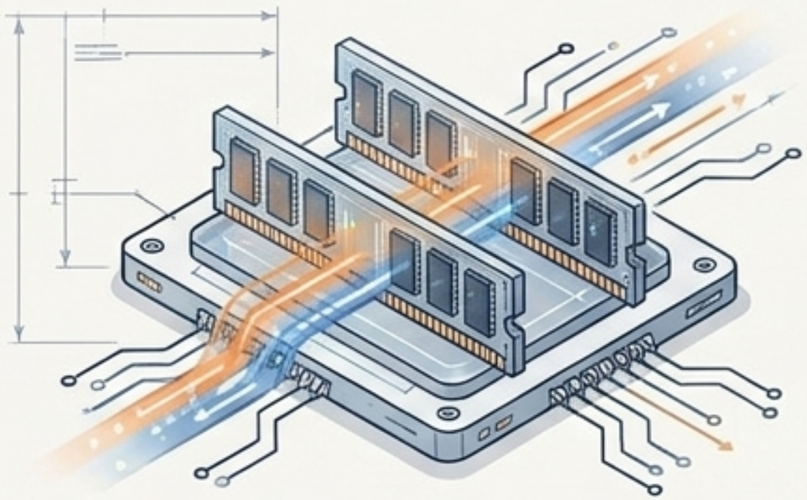


Gemini Notebook

Arquitetura de Memória: Volatilidade vs. Persistência

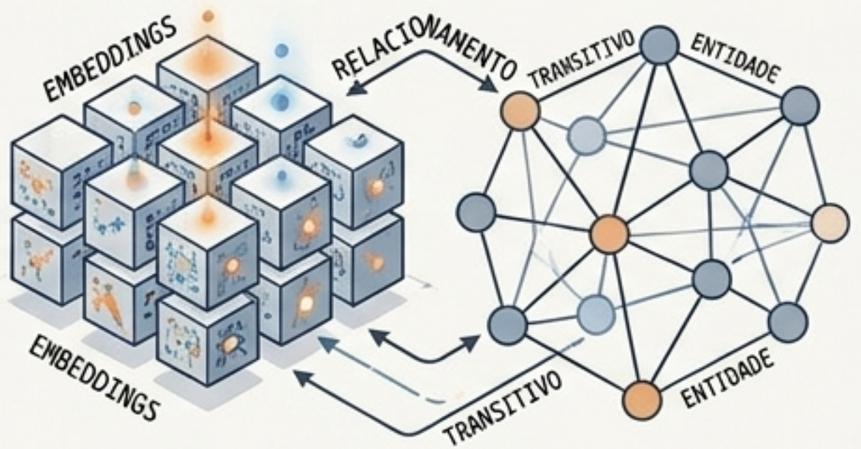
Input do Usuário
e Interações

Curto Prazo (Sessão)



- **Natureza:** Volátil. Histórico da interação atual.
- **Mecanismo:** Buffer de mensagens anexadas (append-only) limitadas pela janela de tokens do LLM.
- **Função:** Preserva o "fio da meada" do raciocínio imediato e contexto tático.

Longo Prazo (RAG Agêntico)



- **Bancos Vetoriais:** Armazenam embeddings para busca semântica. Ideal para fatos históricos e manuais.
- **Bancos de Grafos:** Mapeiam relacionamentos complexos entre entidades. Essencial para consultas transitivas (ex: Amazon Neptune em detecção de fraude).

Gemini Notebook

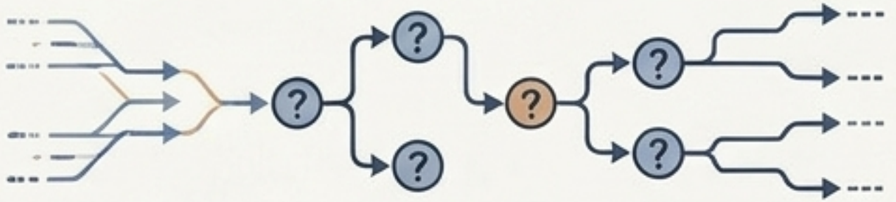
Mecanismos de Planejamento e Raciocínio

1. **ReAct** (Reason + Act)

Raciocínio e ação intercalados em tempo real.
O modelo pensa no passo imediato, age, e reavalia.
Ideal para tarefas dinâmicas e imprevisíveis.

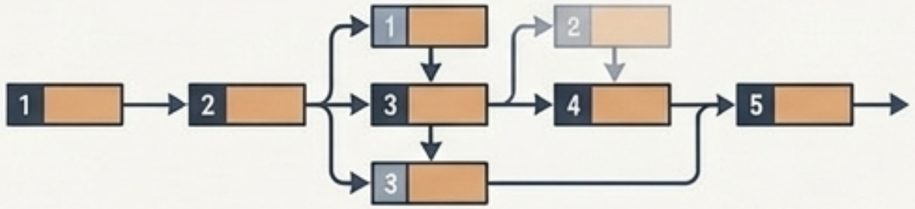
2. Autoquestionamento

Geração explícita de perguntas intermediárias pelo próprio modelo antes de elaborar a resposta final, aprofundando a cadeia de pensamento estruturada.



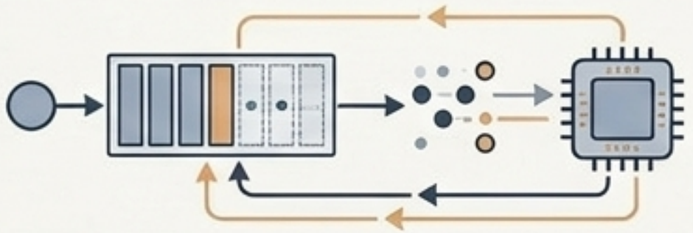
3. Plan-and-Solve

Divisão rígida e antecipada de uma macro-tarefa em subtarefas sequenciais menores. Previne erros de omissão e mantém o agente no trilho.



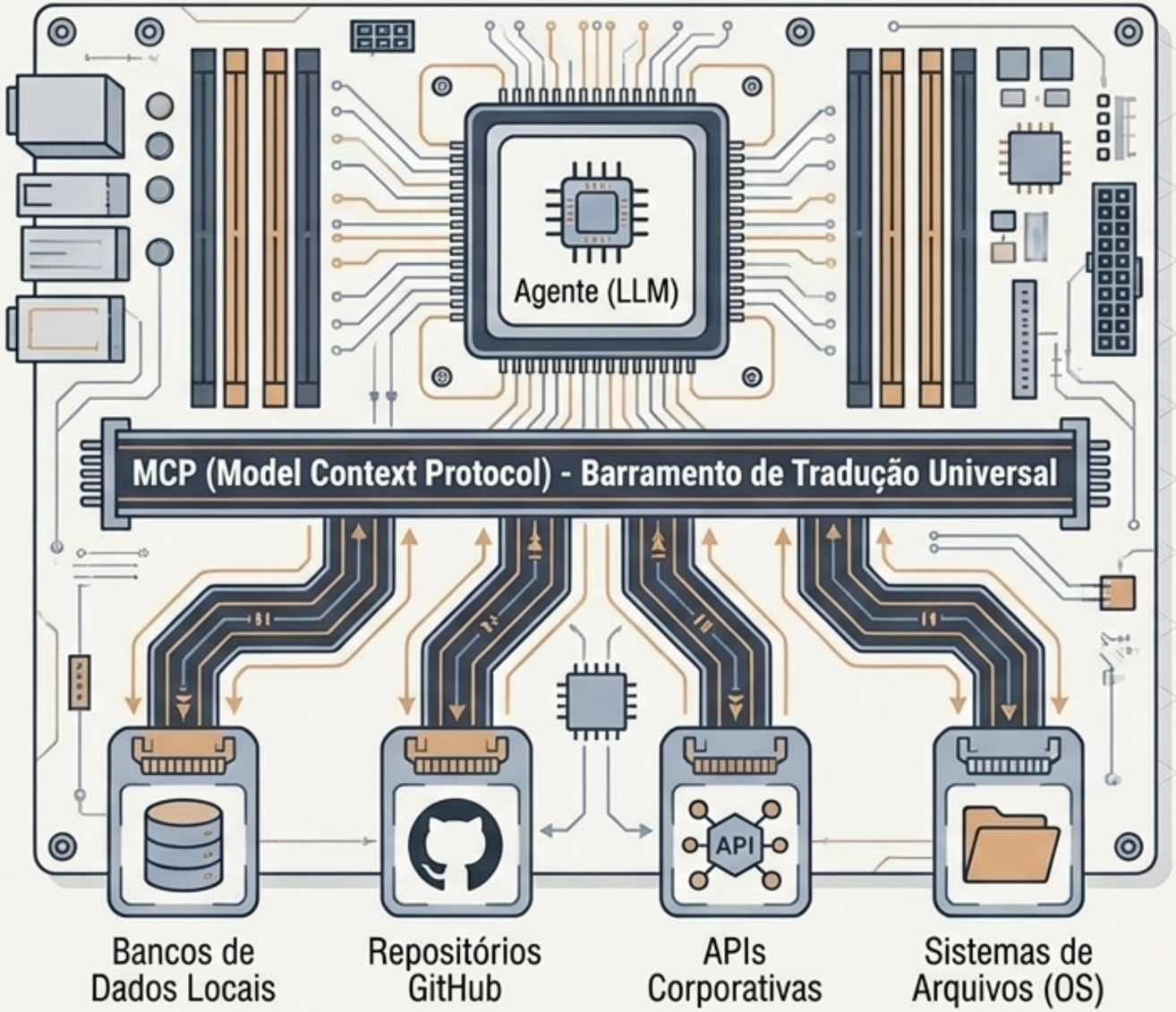
4. **Reflexão** (Autocrítica)

Uso de um buffer de memória episódica onde o agente analisa e avalia criticamente suas próprias decisões passadas para corrigir rotas no próximo turno.



Gemini Notebook

A Linguagem Universal de Ferramentas: Protocolo MCP



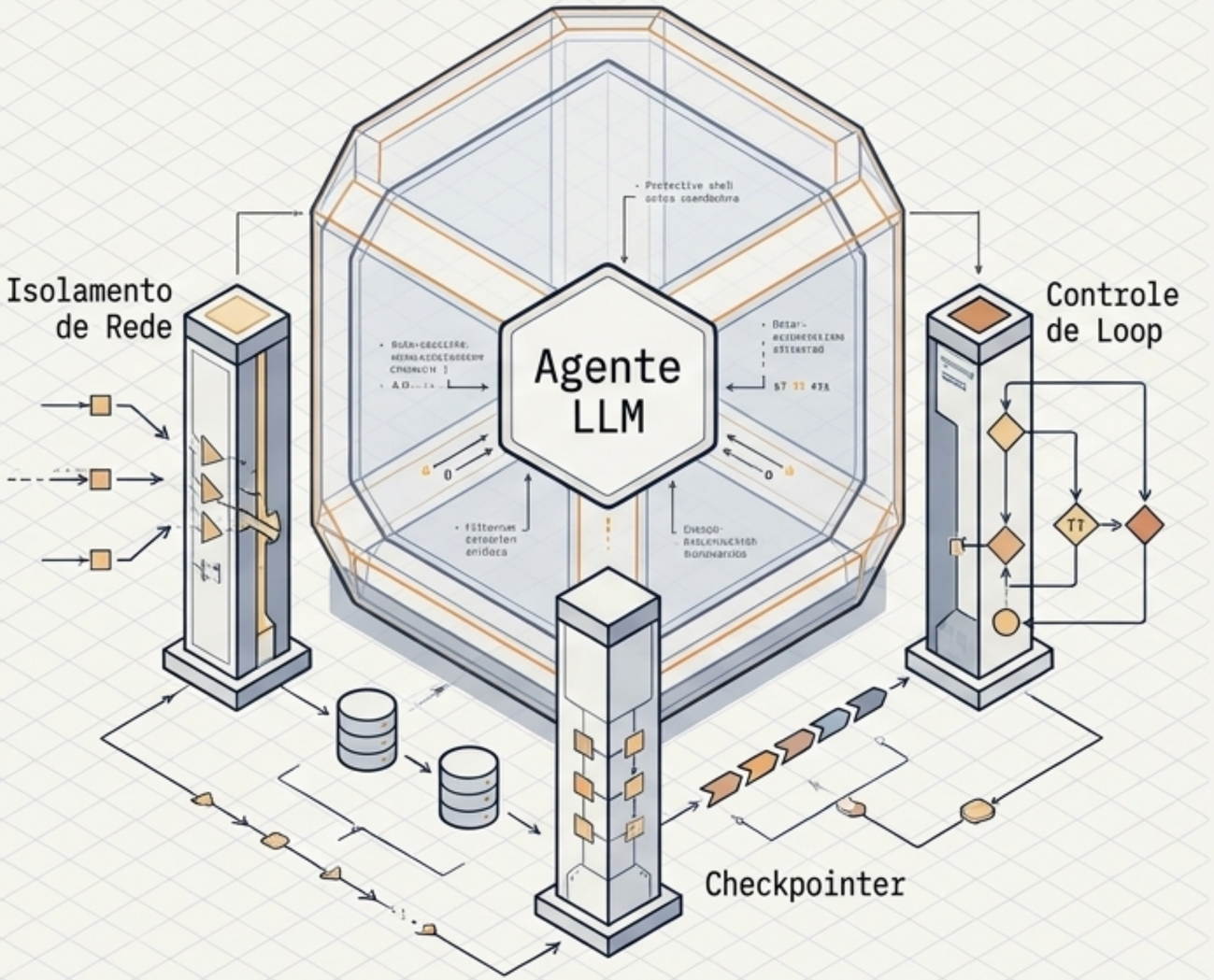
- **O Problema:**
  Historicamente, cada nova ferramenta exigia integrações de código customizadas, criando silos de desenvolvimento intransponíveis.

- **A Solução:**
  Criado pela Anthropic, o MCP atua como um tradutor de código aberto padronizando o ecossistema.

- **O Impacto:**
  Elimina o código de cola. Permite infraestruturas plug-and-play onde agentes leem arquivos locais e APIs nativamente.

Gemini Notebook

Harness Engineering: O Chassi de Execução



O que é o Harness?

É a plataforma de infraestrutura (ex: `LangGraph`, `Amazon Bedrock AgentCore`) onde o agente vive, opera e é gerenciado.

Isolamento e Segurança

Encapsula o agente, separando a lógica de inferência pura do modelo do acesso indiscriminado e arriscado ao sistema operacional base.

Persistência e Checkpoints

Salva o estado da máquina em cada etapa. Permite pausar a execução, recuperar falhas sistêmicas e inspecionar a memória da rodada.

Human-in-the-Loop (HITL)

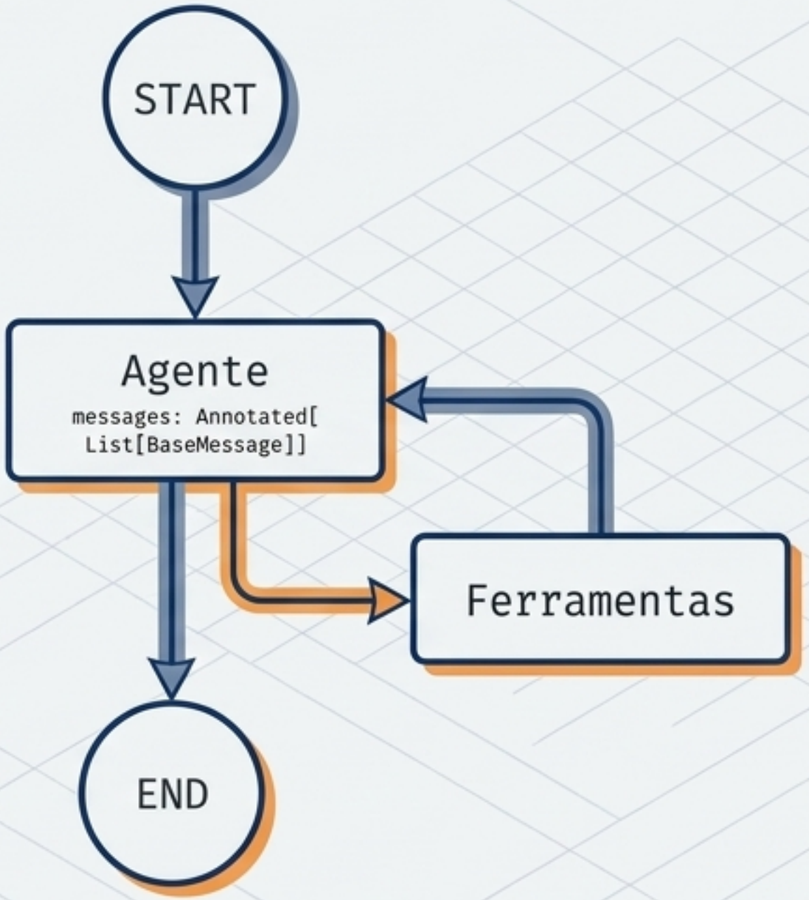
Habilita pontos de parada nativos na infraestrutura, exigindo que um operador humano aprove ou audite ações críticas antes da execução final.

Gemini Notebook

Construção Prática: Orquestrando Máquinas de Estado

```python
class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

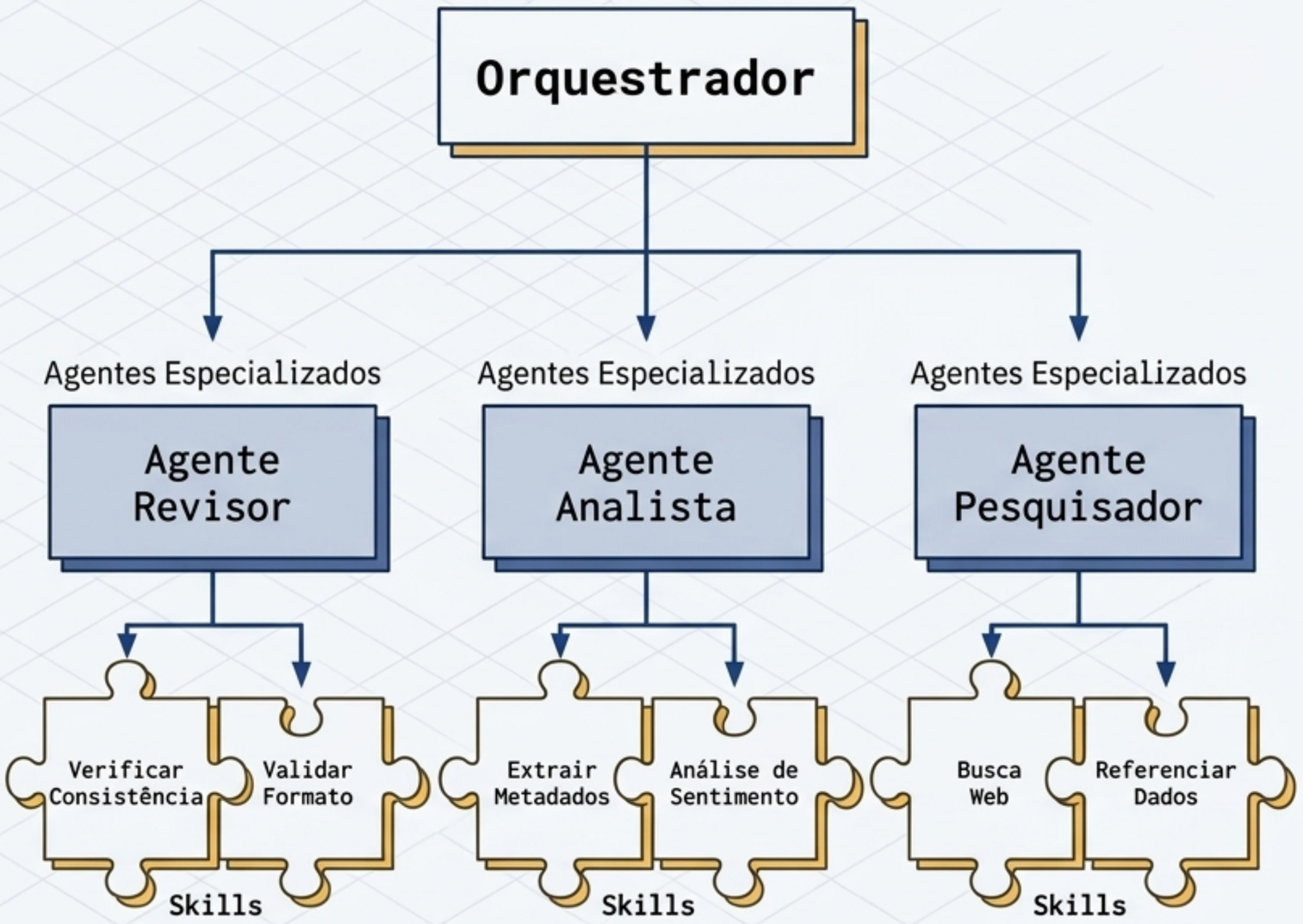
builder = StateGraph(AgentState)
builder.add_node('agente', no_agente)
builder.add_node('ferramentas', ToolNode(ferramentas))
builder.add_conditional_edges('agente', avaliar_passo)
memoria = MemorySaver()
agente = builder.compile(checkpointer=memoria)
```



Síntese Prática: A implementação de agentes em produção não é um “super-prompt”. É a engenharia de um grafo direcionado cíclico gerenciando um estado mutável (TypedDict).

Gemini Notebook

Nível 3 de Autonomia: Sistemas Multiagentes



O **Orquestrador:** Avalia o input do usuário e distribui a carga de trabalho de forma inteligente para os especialistas.

Agentes Especializados: Hiperpecializados em domínios estritos. Possuem instruções dedicadas e contexto limpo,focando em uma fatia isolada do problema.

Skills (Habilidades): Pacotes de instruções e fluxos reutilizáveis. Um agente veste múltiplas skills conectáveis conforme a necessidade tática da rodada.

Subagentes: Operações periféricas delegadas e executadas em paralelo para evitar a poluição do limite de tokens (contexto) do agente principal.

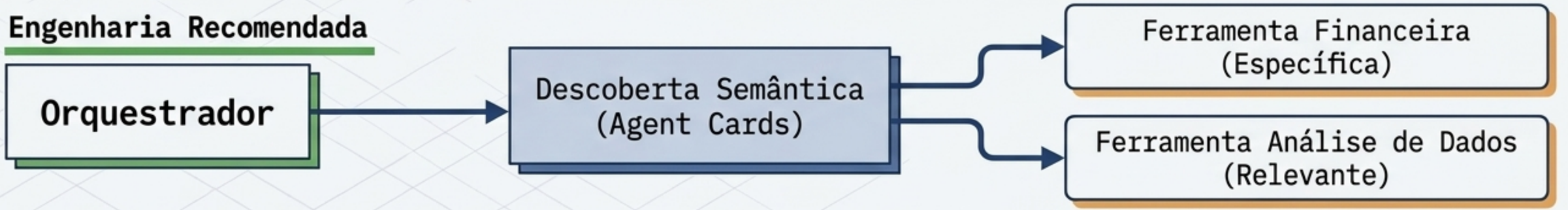
Gemini Notebook

Roteamento e Minimização de Alucinações

Padrão Falho



Engenharia Recomendada

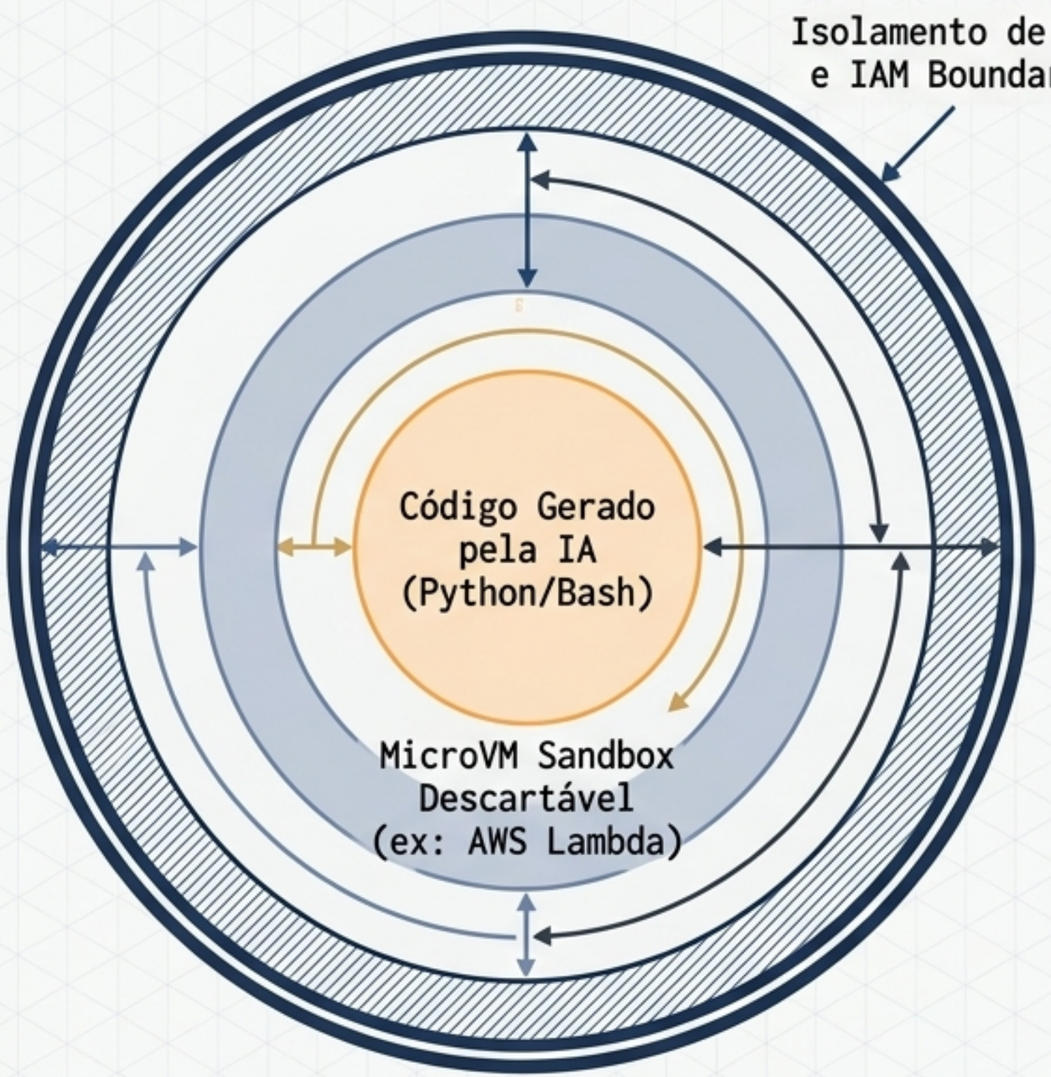


O Problema do Contexto Excessivo:
Fornecer esquemas completos de dezenas de ferramentas polui a janela de contexto do LLM e eleva exponencialmente a probabilidade de chamadas inválidas e erros.

A Solução (Roteamento Semântico):
O orquestrador não repassa payloads completos. Usa 'Agent Cards' (apenas nome e breve descrição). Em tempo de execução, busca semanticamente as ferramentas restritas e injeta no contexto final.

Gemini Notebook

Segurança Zero Trust: Sandboxing para Agentes



O Risco Estrutural:

- Agentes autônomos executam scripts reais.
- Alucinações, erros lógicos ou injeções de prompt maliciosas podem corromper arquivos locais, modificar bases de dados ou vazar credenciais corporativas.

Contenção Física, Não Apenas Semântica:

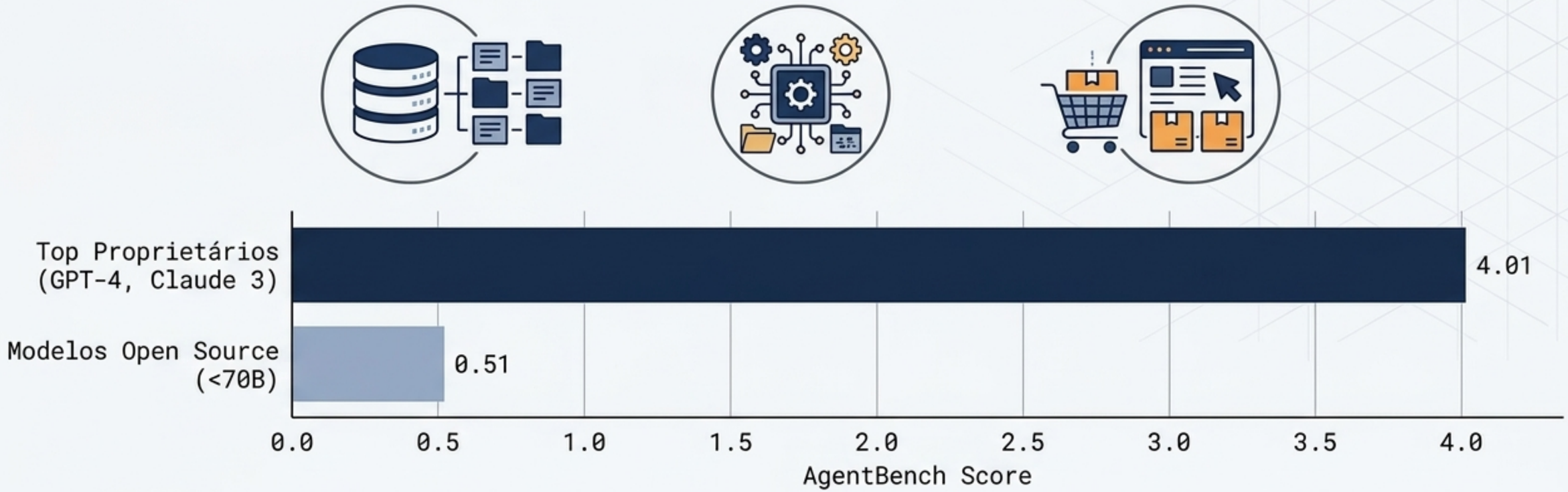
- Não confie exclusivamente nas restrições de prompt ("você não deve apagar arquivos").
- LLMs falham no alinhamento de segurança sob estresse de contexto.

MicroVMs (Máquinas Virtuais Efêmeras):

- Toda execução de código gerado pelo agente deve ocorrer em **sandboxes virtuais descartáveis**.
- Sem acesso à rede principal do desenvolvedor.
- Sem variáveis de ambiente globais.

Gemini Notebook

Avaliação Empírica: A Realidade do AgentBench



O Abismo de Performance	A Limitação do Open Source
Testes em 8 ambientes interativos (OS, Bancos de Dados, Navegação Web) revelam que modelos proprietários via API possuem capacidade real de atuar como agentes práticos.	A esmagadora maioria dos modelos OSS analisados falha criticamente em cenários complexos, provando que excelência em chat não se traduz automaticamente em capacidade agêntica autônoma.

Gemini Notebook

Anatomia das Falhas: Onde e Por Que os Agentes Erram

Tabela de Diagnóstico de Erros

	**Erro TLE: Task Limit Exceeded**	A falha predominante hoje. O agente perde o controle do estado, entra em **recursão profunda** (loops infinitos de geração repetitiva) e excede o limite estrito de rodadas por **incapacidade de formular raciocínio de longo prazo.**
	**Erro IF/IA: Invalid Format & Invalid Action**	**Incapacidade de seguir instruções rígidas de formatação de sistema** (crítico na injeção de parâmetros em Bancos de Dados via SQL) ou seleção de **ações impossíveis fora do espaço vetorial pré-definido pela ferramenta.**
	**O Paradoxo do Código em Treinamento**	Treinar modelos exaustivamente em código melhora a **lógica procedural**, mas os dados sugerem que pode **paradoxalmente prejudicar o pensamento lateral do agente em tarefas de exploração criativa e raciocínio não-computacional.**

Gemini Notebook

O Futuro do Software: Diretrizes de Engenharia

01

Estruture a Memória

- Separe rigorosamente o buffer de contexto da sessão (volátil) da persistência analítica. Utilize bancos de grafos para relacionamentos complexos de entidades.

02

Abrace Protocolos (MCP)

- Interrompa a construção de integrações isoladas em silos. Padronize a conectividade de ferramentas e sistemas de arquivos com o Model Context Protocol.

03

Encapsule a Execução

- Isole o motor do LLM construindo um Harness resiliente (como LangGraph) e obrigue toda execução de código a rodar restrita em Sandboxes imutáveis.

04

Descentralize e Especialize

- Abandone o conceito do 'Agente Tudo-em-Um'. Desenvolva Sistemas Multiagentes arquitetados com Skills estritas, foco isolado e roteamento delimitado semanticamente.

Gemini Notebook